BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

LESSON 217 · Quicksort: tracing and implementation

Name: _____ Class: _____ Date: _____

Work through the pages in order. Answer in BLACK PEN first. Check in GREEN PEN only after the Exit Ticket.

Do Now · Recall previous learning

Q1. After partitioning [8,1,7,3,6] around pivot 6, where is the pivot?

Q2. Why does recursive quicksort need a base case?

Q3. What is the worst-case time complexity of merge sort?

Q4. If the pivot finishes at index 3, give left and right recursive bounds in an array 0..6.

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

I Do · Read new knowledge

Key vocabulary

quicksort — Recursive sort using partitioning around a pivot.

base case — Condition ending recursion.

partition index — Final index occupied by the pivot.

worst case — Input arrangement causing the most expensive run.

recursion stack — Storage used for nested calls.

New learning

Stop small subarrays — When low>=high there are zero or one elements to sort: return immediately.

Partition then divide — Partition places the pivot at its final index p. Do not recursively include index p.

Sort two sides independently — Call QuickSort on low..p−1 and p+1..high. The pivot itself stays fixed.

Worked example

Trace recursive quicksort on [5,1,7,3] (last pivot).

1. Predict the next state — Partition around 3, moving 1 to the left and placing pivot 3 at index 1.

2. Reveal and explain — Left segment [1] is already sorted; sort only the right segment [7,5].

3. Reveal and explain — The right recursion produces [5,7]; final array [1,3,5,7].

We Do · Practise with guidance

Trace recursive quicksort on [6,1,5,2,4].

```
Partition around 4; then sort the left and right sides.
```

Predict the next state

Partition around pivot 4, producing [1,2,4,6,5].

Reveal and explain

The left [1,2] is already sorted; the right [6,5] needs another partition.

After sorting the right part, final array is [1,2,4,5,6].

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

You Do · Work independently

BLACK PEN: Write your own answers. Do not look at the Review & Improve pages yet.

Question 1

Why is if(low>=high) return; required in a recursive C# quicksort?

Question 2

Use last-element Lomuto quicksort on [4,1,6,2]. Give the first partition and the final sorted array.

Question 3

Describe the average and worst-case time complexity of quicksort, why these can differ, and one implementation trade-off relative to merge sort.

Exit Ticket · Answer before checking

After partition, the recursive calls should include the pivot index again.

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

Review & Improve · GREEN PEN

Now change to GREEN PEN. Find each original question. Tick correct answers; read the model and rewrite mistakes.

Page 1 onwards · Do Now

Q1. Index 2 in [1,3,6,8,7].

Q2. To stop on empty or single-element partitions.

Q3. O(n log n).

Q4. Left 0..2, right 4..6; do not include pivot index 3 again.

You Do pages · Independent questions

Q1. [3] Prevents calls on empty [1] or single-element ranges [1], stopping recursion correctly [1].

Q2. [6] Pivot=2 [1]; move 1 left [1] then place pivot to form [1,2,6,4] [2]; recursive right segment [6,4] sorts to [4,6] [1]; final [1,2,4,6] [1].

Q3. [6] Average O(n log n) [1] with balanced partitions [1]; worst $O(n^2)$ [1] with repeated unbalanced partitions [1]; in-place partitioning typically uses less additional array space than merge sort [1], though recursive quicksort still uses call-stack space [1].

Exit Ticket · Model answer

False. The pivot is already fixed and is excluded from both calls.

Try the question again

Choose one question you got wrong. All correct? Choose the hardest.

1. Copy the whole original question:

2. Write a complete new answer in GREEN PEN. Show working if needed:

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

Do Now	I Do	We Do	You Do	Review & Improve
Recall previous learning	Read new knowledge	Practise with guidance	Work independently	Check understanding